

Chapter 1

Introduction to Relational Databases in SQL

1.1 Uniquely identify records with key constraints

1.1.1 Keys and superkeys

What is a key?

- Attributes(s) that identify a record Uniquely.
- As long as attributes can be removed: **superkeys**.
- If no more attributes can be removed: minimal superkey or **key**.

Table 1.1: Car database				
license_no	serial_no	make	model	year
Texas ABC-739	A69352	Ford	Mustang	2
Florida TVP-347	B43696	Oldsmobile	Cutlass	5
New York MP0-22	X83554	Oldsmobile	Delta	1
California 432-TFY	C43742	Mercedes	190-D	99
California RSK-629	Y82935	Toyota	Camry	4
Texas RSK-629	U028365	Jaguar	XJS	4

Note: Table adapted from Elmasri, Navathe (2011): Fundamentals of Database Systems, 6th Ed., Pearson

SK1 = {license_no, serial_no, make, model, year}

SK2 = {license_no, serial_no, make, model}

SK3 = {make, model, year}, SK4 = {license_no, serial_no}, SKi, ..., SKn

A lot of possible superkeys with different combinations. But only four possible **keys**

K1 = {license_no}; K2 = {serial_no}; K3 = {model}; K4 = {make, year}

Here K1 to 3 only consist of one attribute. Removing either "make" or "year" from K4 would result in duplicates. Only one candidate key can be the chosen key.

A simple way of finding out whether a certain column(or combination) contains only unique values, wrap everything within the COUNT() function.

```
SELECT COUNT(column_a, column_b, ...)  
FROM table_name;
```

```
-- then compare the number of rows, if the number of Distinct row matches the total number of
--rows then the column or combination of column can be used as keys
```

```
SELECT COUNT(DISTINCT(column_a, column_b, ...))
```

1.2 Primary keys

- One primary key per database table, chosen from candidate keys
- Uniquely identifies records, e.g. for referencing in other tables
- Unique and not-null constraint both apply
- Should be built from as few column as possible
- Primary keys are time-invariant: choose columns wisely

1.2.1 Specifying primary keys

```
CREATE TABLE products (  
    product_no integer UNIQUE NOT NULL, -- same constraint as of Primary Key  
    name text,  
    price numeric  
);
```

```
CREATE TABLE products (  
    product_no integer PRIMARY KEY,  
    name text,  
    price numeric  
);
```

```
-- Designate composite key as primary key  
CREATE TABLE example (  
    a integer,  
    b integer,  
    c integer,  
    PRIMARY KEY (a, c)  
);
```

Note: Taken from PostgreSQL documentation.

1.2.2 Adding primary key to existing table

```
ALTER TABLE table_name  
ADD CONSTRAINT some_name PRIMARY KEY (column_name)
```

1.3 Surrogate keys

It is an artificial primary key that exist for the sole purpose of being a primary key. Assume that a table consist of only make, model and year column from that table 1.1

1.3.1 Adding a surrogate key with serial data type

Serial data type allow auto increment, and assign automatically when data is input.

```
ALTER TABLE car
ADD COLUMN id serial PRIMARY KEY;
```

1.3.2 Another type of surrogate key

Combine two column to form a new ones for uniquely identifying

```
ALTER TABLE table_name
ADD COLUMN column_c varchar(256);
```

```
UPDATE TABLE table_name
SET column_c = CONCAT(column_a, column_b);
```

```
ALTER TABLE table_name
ADD CONSTRAINT pk PRIMARY KEY (column_c);
```

1.4 Glue together tables with foreign keys

1.4.1 Model 1:N relationships with foreign keys

The next database model

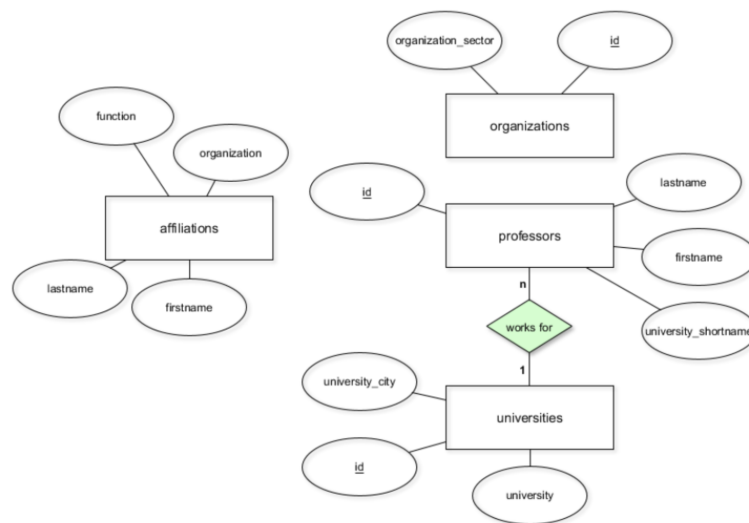


Figure 1.1: ER diagram

In the diagram 1.1 Foreign key will add a one-to-many relationship where one university can have zero or more professors.

Implementing relationship with foreign keys

- A foreign key (FK) points to the primary key (PK) of another table.
- Domain/data type of FK must be equal to domain of PK
- Each value of FK must exist in PK of another table (FK constraint or "referential integrity")
- FK are not actual keys as they can include null and duplicates values.

Specifying foreign keys

```
CREATE TABLE manufacturers (  
    name varchar(255) PRIMARY KEY  
);
```

```
INSERT INTO manufacturers  
VALUES ('Ford'), ('VW'), ('GM')
```

```
CREATE TABLE cars (  
    model varchar(255) PRIMARY KEY,  
    manufacturer_name varchar(255) REFERENCES manufacturers (name) -- or separate column and relationship  
);
```

```
INSERT INTO cars  
VALUES ('Ranger', 'Ford'), ('Beetle', 'VW');
```

--Throws an error!

```
INSERT INTO cars  
VALUES ('Tundra', 'Toyota');
```

Specifying foreign keys to existing tables

```
ALTER TABLE a  
ADD CONSTRAINT a_fkey FOREIGN KEY (b_id) REFERENCES b (id);
```

1.4.2 Model more complex relationships

The final database model

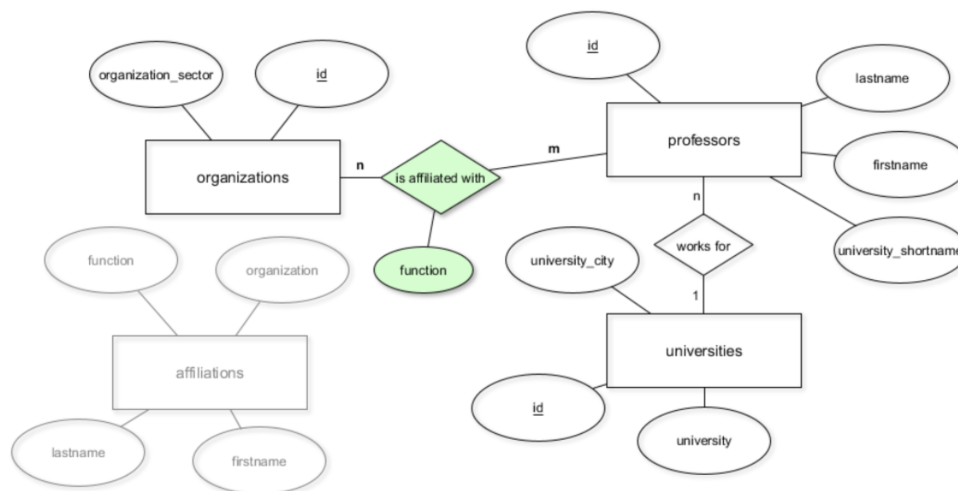


Figure 1.2: Final ER diagram

In this diagram 1.2 Foreign key enable many-to-many relationship between organisation and professors, as professor can be affiliated with more than 1 organisation and organisation itself can be associated with more than 1 professors.

1.4.3 How to implement N:M-relationships

```
CREATE TABLE affiliations (  
    professor_id integer REFERENCES professors (id),  
    organization_id varchar(256) REFERENCES organizations (id),  
    function varchar(256)  
)
```

- Create a table
- Add a foreign keys for every connected table
- Add additional attributes
- No primary key!
- Possible PK = {professor_id, organization_id, function}

1.4.4 Referential integrity

- A record referencing another table must refer to an existing record in that Table
- Specified between two tables
- Enforced through foreign keys

Referential integrity violations

Referential integrity from table A to table B is violated...

- ...if a record in table B that is referenced from a record in table A is deleted
- ...if a record in table A referencing a non-existing record from table B is inserted
- Foreign keys prevent violations!

Dealing with violations

```
CREATE TABLE a (  
    id integer PRIMARY KEY,  
    column_a varchar(64),  
    ...,  
    b_id integer REFERENCES b (id) ON DELETE NO ACTION  
)
```

```
CREATE TABLE a (  
    id integer PRIMARY KEY,  
    column_a varchar(64),  
    ...,  
    b_id integer REFERENCES b (id) ON DELETE CASCADE  
)
```

ON DELETE

- NO ACTION : Throw an error
- CASCADE : Delete all referencing records
- RESTRICT: Throw an error
- SET NULL : Set the referencing column to NULL

- SET DEFAULT : Set the referencing column to its default value, only work if default value is specified

Drop existing constraint

```
--Identify the correct constraint name
SELECT constraint_name, table_name, constraint_type
FROM information_schema.table_constraints
WHERE constraint_type = 'FOREIGN KEY';

--Drop the right foreign key constraint
ALTER TABLE affiliations
DROP CONSTRAINT constraint_name;
```

Chapter 2